

Claude Code Token Optimization Toolkit

Cut your Claude Code token spend 60–90% with 4 free tools — proven on real sessions.

Executive Summary

Claude Code bills on four token buckets: input, output, cache creates, and cache reads. Most users focus on raw input, but output tokens (5× more expensive per token) and cache creates (loaded fresh on every large file read) are the real cost drivers in a typical build session.

This guide covers four tools — Caveman, CLAUDE.md optimization, context-mode, and Codebase Memory MCP — that target each cost driver directly. All four are free and open source. Combined, they produce 85–92% total token reduction on a typical Claude Code session. Real benchmark: a week running Caveman alone cut output tokens 47% on the sessions measured.

Install time: 30 minutes. Payback: immediate.

Table of Contents

- 1. [Why Claude Code Burns Tokens](#)
- 2. [Tool #1: Caveman + CLAUDE.md Optimization](#)
- 3. [Tool #2: context-mode + Codebase Memory MCP](#)
- 4. [Stack These Together: Full Setup in 30 Minutes](#)

Why Claude Code Burns Tokens (And Why Your Bill Looks Like That)

Claude Code is priced on tokens — the chunks of text and data that flow in and out of the model on every exchange. The more tokens, the higher the bill. That part most people understand.

What most people don't understand is *where the tokens actually go*.

The Four Buckets

Every Claude Code session draws from four token buckets, billed at very different rates:

Bucket	What Goes Here	Cost (Sonnet 4.6)
Input	Your messages, file contents, command output	\$3.00 / 1M tokens
Output	Claude's responses	\$15.00 / 1M tokens
Cache Create	Context stored in Anthropic's cache for reuse	\$3.75 / 1M tokens
Cache Read	Re-reading that cached context	\$0.30 / 1M tokens

Output tokens are the most expensive by far — 5× the input rate. But the real drain in a long Claude Code session isn't any single message. It's the *accumulating context* that gets re-loaded on every exchange.

How Context Accumulates

Every time you send Claude Code a message, the model receives:

1. Your full CLAUDE.md (loaded fresh, every single time)
2. Any files it read earlier in the session
3. Every tool call it made — including the complete output of every bash command
4. Its own previous responses

By hour two of a build session, Claude Code isn't just processing your latest question. It's re-reading the last 30,000 tokens of conversation history, 50 lines of CLAUDE.md, and the full output of every `npm install` and test run it ran along the way.

This is why sessions that feel productive get expensive fast. It's not one big cost event — it's death by accumulation.

What Prompt Caching Does (And Doesn't Do)

Anthropic's prompt caching is Claude Code's built-in cost control. When the same context block appears repeatedly, it gets stored and re-read at \$0.30/MTok instead of \$3.00/MTok — a 10× cost reduction on repeated input.

This is why a typical optimized Claude Code usage pattern shows ~96% of tokens as cache reads. The caching is working. But two things it doesn't help with:

1. **Output tokens** — every response Claude generates is billed at \$15/MTok regardless. If Claude explains what it's doing in 300 words, that's about 225 output tokens, billed at over \$0.003 per response. Multiply by 100 exchanges in a session and it adds up.
2. **Cache creates** — the first time a large block of context is loaded, it gets cached at \$3.75/MTok. Your CLAUDE.md, every file Claude reads for the first time, every long command output — those are all cache creates. They're cheaper than raw input on repeat, but expensive on first load.

The Three Levers

Once you see the four buckets, the optimization strategy becomes obvious:

Lever 1: Shrink what gets loaded every message. Your CLAUDE.md is re-loaded on every single exchange. A 852-token CLAUDE.md costs 2.8× more per message than a 300-token one — before you've written a single line of code. Stripping it down is the highest-ROI, zero-tool change you can make.

Lever 2: Stop output tokens from inflating. Claude Code defaults to being thorough: explaining reasoning, summarizing what it did, restating the question. Most of that prose is invisible overhead — you read the code, not the explanation. The Caveman skill rewrites Claude's output style to be maximally terse without losing technical accuracy. Output tokens drop 50–75% on prose-heavy responses.

Lever 3: Stop large outputs from hitting the context window raw. When Claude reads a 500-line file or runs a noisy build command, the full output lands in context — even the parts that are irrelevant. Tools like context-mode intercept large outputs and return summaries, and Codebase Memory MCP replaces repeated file reads with a knowledge graph query. Instead of re-reading a 2,000-token file, Claude queries a graph node.

What This Guide Covers

The next four sections cover the tools that target each lever, in the order you should install them:

- 1. **CLAUDE.md Optimization** — the zero-tool win (Lever 1)
- 2. **Caveman** — terse output, fewer output tokens (Lever 2)
- 3. **context-mode** — sandbox large outputs (Lever 3)
- 4. **Codebase Memory MCP** — replace file reads with graph queries (Lever 3)

Each section includes real before/after token counts from actual Claude Code sessions, the exact install steps, and a runnable demo script you can use to reproduce the benchmark.

By the end, you'll have a stack that produces 60–90% less token consumption on a typical build session — not a marketing estimate, but a number you can verify yourself with ccusage.

Tool Breakdown: Caveman + CLAUDE.md Optimization

These two tools target your two biggest cost levers and cost nothing to run. Install both before touching anything else.

Tool #1: Caveman

What it does: Rewrites Claude's own output to be maximally terse — shorter sentences, no filler, no explanatory prose you didn't ask for. Code output untouched. Only narrative responses compress.

GitHub: <https://github.com/JuliusBrussee/caveman>

Cost: Free

Install time: ~2 minutes

Real Before/After

Measured across two consecutive Claude Code days (same project, same model — Sonnet 4.6):

Day	Caveman	Output tokens	Session cost
Jun 9 (baseline)	OFF	52,759	\$4.23
Jun 10 (Caveman active)	ON	27,794	\$3.96

Output tokens down 47%. Note: Jun 9 was a heavier session (scaffolding a full project + writing Linear issues), so this isn't a clean A/B. The honest range based on community benchmarks is 40–65% reduction on output prose across typical sessions.

What doesn't change: code blocks, tool call results, file content. Caveman only shrinks the conversational wrapper — the "here's what I'm doing" and "I'll now proceed to" text that pads every response but adds no value.

Install

```
curl -fsSL https://raw.githubusercontent.com/JuliusBrussee/caveman/main/install.sh |
bash
```

This wires Claude Code hooks, a stats tracker, and a status line badge automatically. No manual config needed.

Activate

Start any Claude Code session and run:

```
/caveman
```

Or say "caveman mode" in the chat. Active for that session. Hooks keep it persistent across sessions once installed.

Intensity levels

Three levels — `lite` , `full` (default), `ultra` . Full drops articles and filler. Ultra adds abbreviations and removes conjunctions. Start with full; drop to lite if you find it breaks your explanations.

```
/caveman lite      # tight but readable
/caveman full      # classic caveman (default)
/caveman ultra     # maximum compression
```

Biggest limitation

Caveman only affects output tokens, which are ~0.67% of total session token volume in a well-optimized setup. If you're already running heavy prompt caching (cache reads >90% of your traffic), Caveman's absolute dollar impact is modest. On a \$9/week spend, 47% output reduction saves roughly \$1.50–\$2.50/week — not transformative on its own, but it stacks with the other tools and compounds over months.

Tool #2: CLAUDE.md Optimization

What it does: Shrinks your CLAUDE.md to the minimum that Claude actually needs. Every excess token in CLAUDE.md is loaded into every single message of every session — it's the highest-leverage, zero-dependency change you can make.

Cost: Free. No install. One hour of editing.
Impact: Permanent — applies before any tool, every session.

Real Before/After

Measured on an active project CLAUDE.md (`halle-b-pipeline`):

Metric	Before	After target
Words	1,136	~400
Est. tokens	~852	~300
Reduction	—	~65%

At ~107,000 cache-create tokens per week (your baseline), a 65% CLAUDE.md reduction on every message materially cuts cache-create costs across every session, permanently.

How to audit yours

```
# Clone this repo and run:
bash demos/benchmark_claudemd.sh /path/to/your/CLAUDE.md
```

Output shows current word count, estimated token count, and how far over the 300-token target you are.

What to strip

Most bloated CLAUDE.md files contain three categories of dead weight:

- 1. Things Claude already knows from the code** Remove: file structure descriptions, "we use React" or "this is a Node project" — Claude reads the repo. Keep: constraints that aren't obvious from the code (e.g., "never use `var`", "all dates in UTC").
- 2. Process descriptions that should be in issues** Remove: multi-paragraph explanations of your workflow, team conventions, how PRs work. Keep: one-line rules that apply every session (e.g., "always run tests before committing").
- 3. Aspirational rules you don't enforce** If it's not a rule you'd stop a PR for, it doesn't belong in CLAUDE.md. Every soft preference loaded into context is tokens spent on guidance Claude will occasionally ignore anyway.

Target format

```
# [Project Name]

## Rules
- [hard constraint — one line]
- [hard constraint — one line]

## Stack
- [key tech only — what matters for how Claude should write code]

## Never
- [explicit prohibitions only]
```

Under 300 tokens. Claude doesn't need more than this to do good work — the codebase tells the rest of the story.

Demo

```
bash demos/benchmark_claudemd.sh ~/your-project/CLAUDE.md
```

Run before and after editing to see your actual reduction.

What These Two Tools Stack To

Tool	Output token impact	Cache create impact	Est. weekly savings
------	---------------------	---------------------	---------------------

Caveman	-47% on output	None	~\$1-2/week
CLAUDE.md optimization	None on output	-65% on CLAUDE.md portion of cache creates	~\$1-2/week
Combined			~\$2-4/week on a \$9 baseline

Next section covers context-mode and CBM — the tools that address cache creates from large file reads and command output, which are the remaining cost driver.

Tool Breakdown: context-mode + Codebase Memory MCP

These two tools target cache creates — what gets loaded into context when Claude reads files or processes large outputs. Together they address the second-largest cost driver after output tokens.

Tool #3: context-mode

What it does: Intercepts large tool outputs — file reads, search results, build logs — and returns a compressed summary instead of the raw content. Claude still gets the information; your context window doesn't pay for 800 lines of `package-lock.json`.

GitHub: <https://github.com/sgaabdu4/claude-code-tips>

Cost: Free

Install time: ~5 minutes

Real Before/After (community benchmark)

Measured by sgaabdu4 across 15 controlled Claude Code sessions:

Scenario	Without context-mode	With context-mode	Reduction
Large file read (500+ lines)	~8,000 tokens	~160 tokens	98%
npm install output	~3,200 tokens	~200 tokens	94%
Test suite output (50 tests)	~5,500 tokens	~300 tokens	95%

Note: These are tool-output tokens, which feed into cache creates. The 98% figure is real — but only applies when a large output is what would have hit your context. On a session that reads mostly small files or has few bash commands, the impact is smaller.

Install

```
# Clone the tips repo
git clone https://github.com/sgaabdu4/claude-code-tips ~/.claude/tips
```

```
# Add to your CLAUDE.md or Claude Code settings:
# Use context-mode for all tool outputs > 200 lines
```

Full config instructions at the repo's README. The key setting is the line threshold — outputs below the threshold pass through unchanged; outputs above get summarized.

How it works

context-mode hooks into Claude Code's tool execution pipeline. When a bash command or file read returns more than the configured line threshold, context-mode intercepts the output, passes it through a local summarization step, and hands Claude a structured summary instead of the raw text.

Claude gets: "npm install succeeded, 847 packages, 3 peer dependency warnings (listed)."
Context window gets: ~180 tokens instead of ~3,200.

Biggest limitation

Summaries can miss edge-case details. If you're debugging a subtle issue buried in line 743 of a log file, you'll want to run without context-mode to see the raw output. Toggle off for debugging sessions; toggle on for building sessions.

```
# Disable for current session
context-mode off

# Re-enable
context-mode on
```

Tool #4: Codebase Memory MCP (CBM)

What it does: Builds a persistent knowledge graph of your codebase. Instead of re-reading files from disk every session, Claude queries the graph — one graph lookup costs a fraction of a full file read.

GitHub: <https://github.com/sgaabdu4/claude-code-tips> (same repo, different module)

Cost: Free

Install time: ~10 minutes (plus first-run indexing: 5–20 min depending on codebase size)

Real Before/After (community benchmark)

Metric	Without CBM	With CBM (warm)	Reduction
"Where is X defined?"	~12,000 tokens (reads 8 files)	~400 tokens (graph query)	97%
Session 2 cold start tokens	baseline	–60–80% of file-read input	60–80%
First session (indexing)	baseline	+20% (index cost amortized over future sessions)	—

The big win is session 2+. First session pays an indexing cost. Every subsequent session reads from the graph — dramatically cheaper.

Install

```
# Add CBM MCP to your Claude Code config:
claude mcp add codebase-memory-mcp

# First session in a new project – build the index:
/cbm:index

# Subsequent sessions – Claude auto-queries instead of reading files
```

How it works

CBM indexes your codebase into a graph: nodes are files, functions, classes, and symbols; edges are imports, calls, and references. When Claude needs to find where `UserService` is defined, instead of opening files one by one, it queries: "node type=class name=UserService" and gets back the file path + line number in one small response.

For large codebases with many sessions, this is the highest-leverage tool in the stack.

When it matters most

CBM pays off most on:

- **Multi-session builds** — the index persists across sessions; the cost is paid once
- **Large repos** (500+ files) — more files = more file reads Claude would otherwise do
- **Refactoring sessions** — lots of "where is X used?" queries

For small single-session scripts or notebooks, skip CBM. Indexing cost won't pay back.

Biggest limitation

CBM index goes stale when code changes significantly. Run `/cbm:reindex` after major refactors. On fast-moving codebases, you'll reindex every few sessions — low cost, but worth knowing.

What All Four Tools Stack To

Tool	Primary target	Est. reduction on target
CLAUDE.md optimization	Cache creates (every message)	65%
Caveman	Output tokens	40–65%
context-mode	Cache creates (large outputs)	94–98%
CBM	Cache creates (file reads, session 2+)	60–97%

Full stack measured result (community, 15-session average): **85–92% total token reduction** vs. vanilla Claude Code on a typical build session.

Next: the "Stack These Together" setup guide — exact install order and the one config that makes all four play nicely.

Stack These Together: Full Setup in 30 Minutes

Install in this exact order. Each tool is independent but the order matters for measurement — you want baseline data before adding anything.

Step 1: Measure baseline (~5 min)

Install ccusage first. Run for at least one normal session before touching anything else.

```
npm install -g ccusage
ccusage report          # see your current weekly spend + token breakdown
```

Note your weekly output tokens and cost. This is your "before" number. Everything else measures against it.

Step 2: Strip your CLAUDE.md (~20 min, one-time)

No install. Just edit.

```
# Audit first — see how bad it is
bash demos/benchmark_claudemd.sh /path/to/your/CLAUDE.md
```

Target: under 300 tokens (~400 words). Strip everything that falls into these three categories:

1. **Things Claude reads from the code** — file structure, framework choices, obvious conventions
2. **Process docs** — workflow descriptions, PR procedures, team norms
3. **Soft preferences** — anything you wouldn't block a merge over

Keep: hard constraints, explicit prohibitions, one-line rules that apply every session.

Run the audit script again after editing to confirm you hit the target.

Why do this first: CLAUDE.md tokens load on every single message. A 65% reduction here is permanent and compounds across every tool that follows.

Step 3: Install Caveman (~2 min)

```
curl -fsSL https://raw.githubusercontent.com/JuliusBrussee/caveman/main/install.sh |
bash
```

Activate at the start of each Claude Code session:

```
/caveman
```

Or say "caveman mode". The hooks auto-persist it across sessions once installed.

Run `ccusage report --compare` after one day to see your output token delta.

Step 4: Install context-mode (~5 min)

```
git clone https://github.com/sgaabdu4/claude-code-tips ~/.claude/tips
```

Follow the README to set your line threshold (recommended: 200 lines). Any tool output above that threshold gets summarized before hitting context.

Test it: run a command that produces long output (`cat a-large-file.txt`) and watch the token count in ccusage. It should drop significantly vs. the same command without context-mode.

Step 5: Install CBM (~10 min)

```
claude mcp add codebase-memory-mcp
```

First session in each project:

```
/cbm:index
```

This builds the knowledge graph. Takes 5–20 minutes depending on repo size. Pay the cost once; every session after reads from the graph.

After indexing, Claude will automatically query the graph instead of reading files. You'll see the `file read` tool calls drop significantly in your session logs.

Verify the Stack

After running all four tools for one full week:

```
ccusage report --compare
```

Compare to your baseline from Step 1. You should see:

- **Output tokens:** –40–65% (Caveman)
 - **Cache creates:** –50–80% (CLAUDE.md + context-mode + CBM combined)
 - **Total cost:** –40–70% depending on your session mix
-

The Cumulative Number

Based on community benchmarks across 15 controlled sessions using the full four-tool stack:

85–92% total token reduction vs. *vanilla Claude Code on a typical build session.*

Your number will vary based on session type. Heavy-file sessions see bigger gains from context-mode and CBM. Output-heavy sessions see bigger gains from Caveman. CLAUDE.md optimization is always the highest ROI per minute spent.

Quick Reference: What Each Tool Targets

Tool	Install	What It Shrinks	When It Matters
CLAUDE.md optimization	Manual edit	Cache creates (every message)	Always
Caveman	2 min script	Output tokens	Every session
context-mode	5 min clone	Cache creates (large outputs)	File/command-heavy sessions
CBM	10 min + index	Cache creates (file reads)	Multi-session builds, large repos

Troubleshooting

Caveman not activating: Run `/caveman` explicitly in session. Check hooks installed in `~/.claude/hooks/`.

context-mode summarizing things it shouldn't: Lower the line threshold or toggle off for debugging:
`context-mode off`.

CBM index stale after refactor: Run `/cbm:reindex`. Takes same time as initial index.

ccusage showing no improvement: Check one tool at a time. Run `ccusage report --compare` after each addition. One tool may not be wired correctly.